INDEX

NAME: Hemsanjay.D ROLL NO.: 2407011911

STD.: CSE DIV./SEC.: SUBJECT: Computer Network

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1)	27/7/26	Various Network commands in Linux & windows.		
2)	28/7/26	Different types of Network cables.		
3)	3/8/26	Packet tracer tool Installation + interface overview.		
4)	10/8/26	Setup + configure LAN using a switch + ethernet in your lab.		
5)	17/8/26	Experiments on Packet capture tool: wireshark		
6)	24/8/26	write a program to implement error detection + correction using HAMMING code		
7)	7/9/26	write a program to implement how control at data link layer using sliding window protocol.		
8)	21/9/26	Nmap Live host discovery using TryHackme		
9)	28/9/26	Implementation of Subnetting in cisco packet Tracer.		
10)	29/9/26	Internetworking with routers in cisco packet Tracer.		
11)	5/10/26	Simulation of RIP using cisco packet tracer.		
12)	6/10/26	End-End communication at TL		

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
13)	6/10/26	Implement ping program.		
14)	10/10/26	code using RAW sockets to implement packet sniffing.		

Completed
13/10/26

Ex. No. 1 Various Network commands used in Linux

* Aim:-

To study the various network commands used in Linux and windows.

* commands:-

1) arp -a:-

ARP is short form of address resolution protocol. It will show the IP address of your computer.

=> O/P:-

-gateway (10.0.2.2) at 52:55:0a:00:02:02 [ether] on enp0s3.

2) hostname:-

It simply displays the name of the your computer.

=> O/P:

fedora.

3) netstat:-

netstat displays a variety of statistics about a computers active TCP/IP connections.

=> O/P:-

Unix 3 [] STREAM CONNECTED 12426.

4) ifconfig:

This command displays detailed config information about your TCP/IP connection including router, gateway, DNS, DHCP & type of ethernet adapter in your system:-

=> O/P:-

enp0s3 flags=4163<UP, BROADCAST, RUNNING, MULTICAST> mtu 1500

inet 10.0.2.15 netmask 255.255.255.0 broadc

5) nslookup:-

It is a tool used to perform DNS lookups in Linux. It is used to display DNS details, such as the IP address of particular computer.

=> eg:- nslookup www.google.com

=> O/P:- Server: 127.0.0.53

Address: 127.0.0.53#53

Non-authoritative answer.

Name: www.google.com

Address: 142.251.152.119.

* Result :-

Thus, the various commands in Linux and windows are executed.

Ex. NO. 2 Different types of Network cables.

* Aim:-

Study of different types of computer Network cables.

(a) understand different types of network cable:-

Different type of cables used in networking are:-

(1) Unshielded Twisted Pair (UTP) cable.
(2) Shielded Twisted Pair (STP) cable.
(3) coaxial cable.
(4) Fibre optic cable.

Cable Type	category	Maximum Data Transmission	Advantage/ Disadvantage	Application /use
UTP	category 3	10 Mbps	*Advantage:- • cheaper cost • Easy to install as they have a smaller overall diameter *Disadvantage: • More prone to EMI and noise	10Base-T ethernet
	category 5	upto 100 Mbps		Fast ethernet, Gigabit ethernet
	Category 5e	1 Gbps		Fast ethernet Gigabit ethernet.
STP	category 6, 6a	10 Gbps	*Advantage: • Shielded • Faster than UTP *Disadvantage: • Expensive. • Greater install effort.	Gigabit ethernet, 10 G ethernet widely used
SSTP	category 7	10 Gbps		Gigabit ethernet 10 G ethernet (100 m)

Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	*Advantage:- • High bandwidth • Immune to interference. • Versatile. *Disadvantage:- • Limited distance • Cost. • Size is bulky.	Speed of signal 500m televisi network high spee internet connectio
Fibre optics cable	Single mode Multi mode	1000Mbps	*Advantages:- • High speed. • High bandwidth • High Security. • Long distance. *Disadvantages: • Expensive. • Requires skilled installers.	Maximu distance fibre op cable i around 100m.

(b) Make your own Ethernet cross-over cable

Tools & parts needed:

- Ethernet cabling. CAT5e is certified for gigabit support.
- A crimping tool. This an all-in-one networking tool shaped to push down the pins in the plug.
- Two RJ45 plugs.
- optional two plug shields.

*Step 1:-

To start construction of the device, begin by threading shields onto the cable.

* Step 2:

Next, strip approx 1.5cm of cable shielding from both ends. The crimping tool has a round area to complete this task.

* Step 3:

After, you will need to untangle the wires; there should be four "twisted pairs". Referencing back to the sheet, arrange them from top to bottom. One end should be in arrangement A and the other in B.

* Step 4:-

once the order is correct, bunch them together in a line, and if there are any that stick out farther than others, snip them back to create an even level. The different aspect is placing these into the RJ45 plug without messing up the order and the gold pins facing forward you.

* Step 5:-

Next, push the cable right in. The notch at the end of the plug needs to be just over the cable shielding, and if it isn't, that means that you stripped off too much shielding. simply snip the cables back a little more.

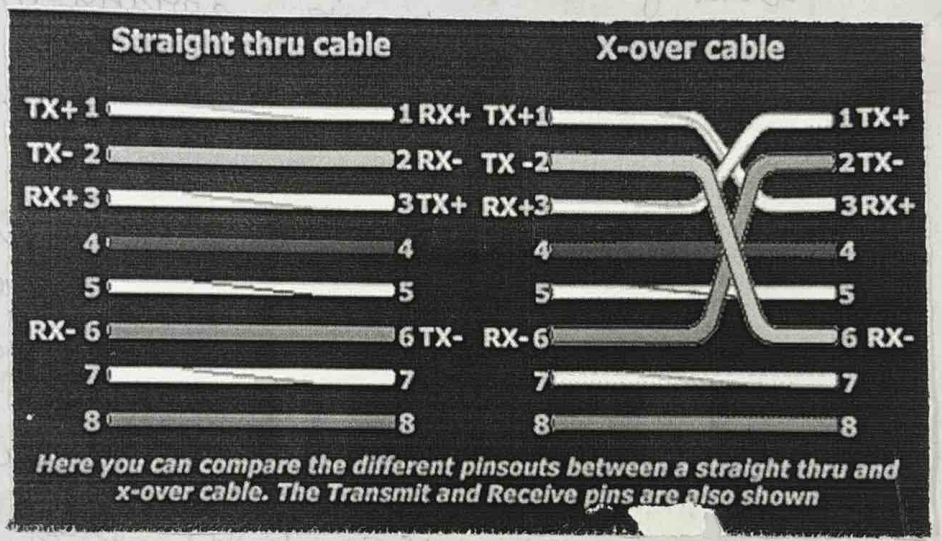
* Step 6:-

After the wires are securely sitting inside the plug, insert it into the crimping tool and push down.

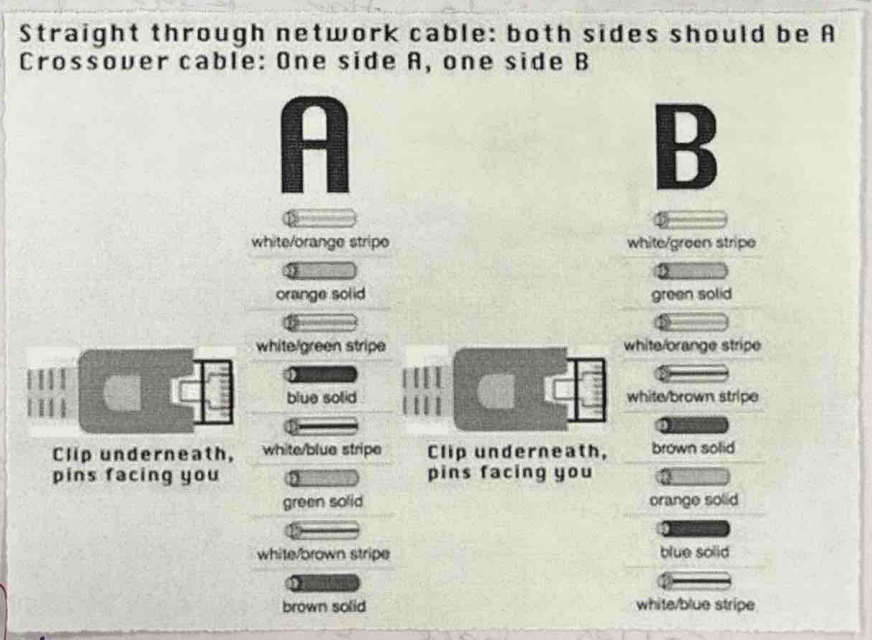
* Step 7:-

Lastly, repeat for the other end using diagram B.

To test it, plug it in & attempt to connect two devices directly.



Here you can compare the different pinsouts between a straight thru and x-over cable. The Transmit and Receive pins are also shown



* Result:

Thus, Different types of network cables is studied.

Ex. No. 3 The Packet tracer tool Installation and user Interface overview.

* Aim:-

To understand environment of Cisco Packet tracer tool installation and user interface overview.

* Introduction:-

A simulator, as the name suggests, simulates network devices and its environment.

1) It allows you to model complex systems without the need for dedicated equipment.
2) It helps you to practice your network configuration and troubleshooting skills.
3) It is available for both the Linux + windows desktop environments.

* Installing Packet tracer:

To download packet tracer, go to https://www.netacad.com + log in with your cisco networking academy credentials; then click on the Packet Tracer graphic and download.

=> windows:

Installation in windows is pretty simple and straightforward; the setup comes in a single file name PacketTracer_setup6.0.1.exe. open this file to begin the setup.

→ Linux:

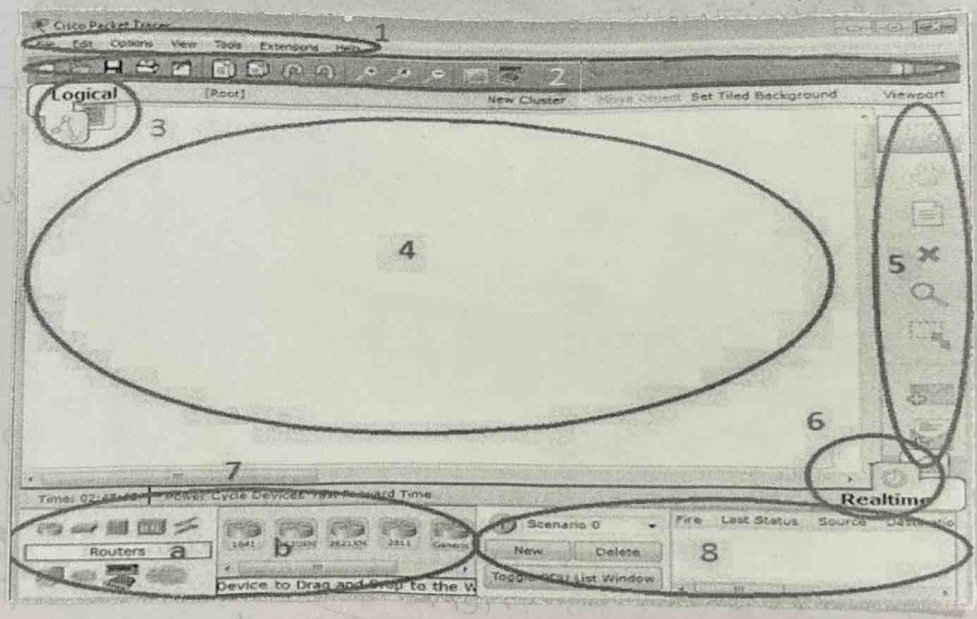
Linux users with an Ubuntu distribution should download the file for ubuntu and those using fedora must download the file for fedora. Grant executable permission to this file by using chmod.

chmod +x PacketTracer601_i386-installer-rpm.bin

./PacketTracer601_i386-installer-rpm.bin

* User interface overview:-

The layout of packet Tracer is divided into several components. The components of the packet tracer inteface as follows:



1) Menu bar:-

This is a common menu found in all software applications.

2) main toolbar:-

This bar provides shortcut icons to menu options that are commonly accessed, such as open, save, zoom, undo & redo.

3) Logical / physical workspace tabs:-
These tabs allow you to toggle between the Logical & physical work areas.

4) workspace:-
This is the area where topologies are created & simulations are displayed.

5) common toolsbar:-
This toolbar provides controls for manipulating topologies, such as select, move layout, place note, delete.

7) Real-time / Simulation tabs:-
These tabs are used to toggle between the real & simulation modes.

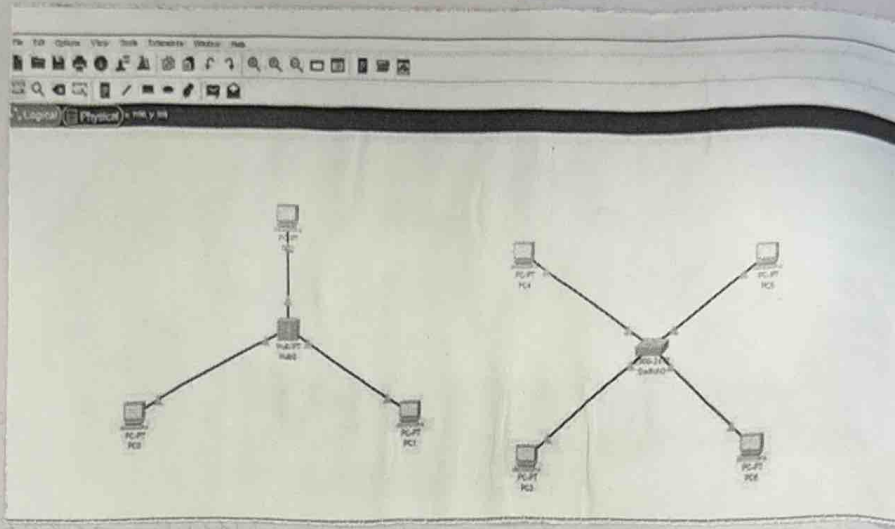
8) User-created packet box:-
users can create highly-customized packets to test their topology from this area and the results are displayed as a result.

* Analyse the behaviour of network devices using CISCO packet tracer simulator.

1) From the network component box, click & drag & drop the below components:
   (a) 4 Generic PCs & one HUB.
   (b) 4 Generic PCs & one switch.

2) click on connections:
(a) click on copper straight-Through cable.
(b) Select one of the PC & connect it to HUB using the cable. similarly connect remaining 3PCs to the HUB.
(c) similarly connect 4PCs to the switch using copper straight-through cable.

3) click on the PCs connected to hub go to Desktop tab, click on IP config & enter an IP address & subnet mask



click on the PDU (message icon) from the common tool bar.

(a) Drag & drop it on one of PC & then drop it on another PC connected to the HUB.

4) Observe the flow of PDU from source PC to destination PC by selecting the realtime mode.

5) Repeat step 3 to step 5 for the PCs connected to switch.

6) Observe how HUB & switch are forwarding the PDU & write your observation & conclusion about the behaviors of switch & HUB.

* Result:

Thus, the packet tracker tool is installed successfully.

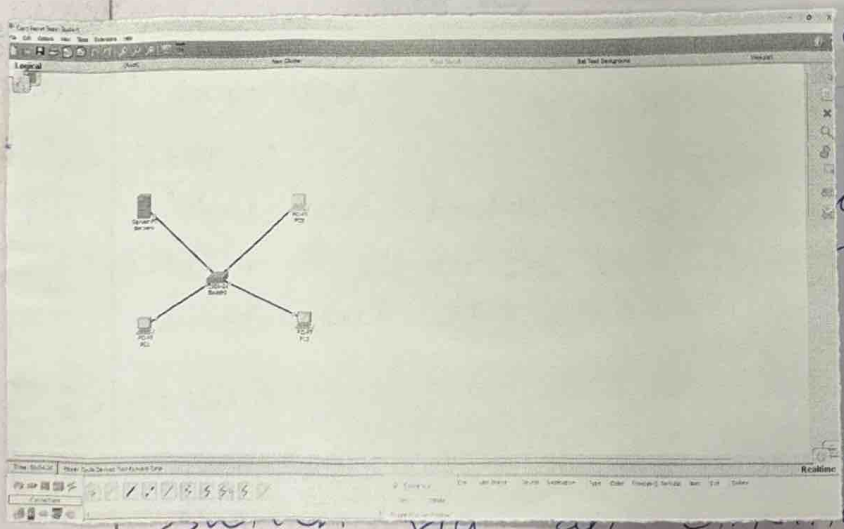
EX.NO.4 Setup & configure LAN (Local area network) using a switch & ethernet cables in your lab.

* Aim:
To Setup & configure LAN using a switch & ethernet cables in your lab.

* LAN:
LAN refers to a network that connects devices within a limited area, such as an office building, school. It enables users to share resources, including, data printers. A LAN switch serves as the primary connecting device.

* How to set up a LAN:-

• Step 1:



ate network

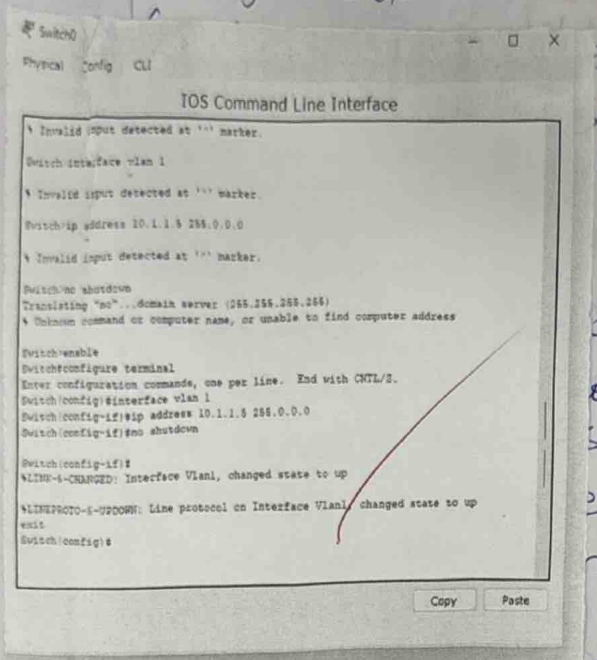
s, a switch
ich is

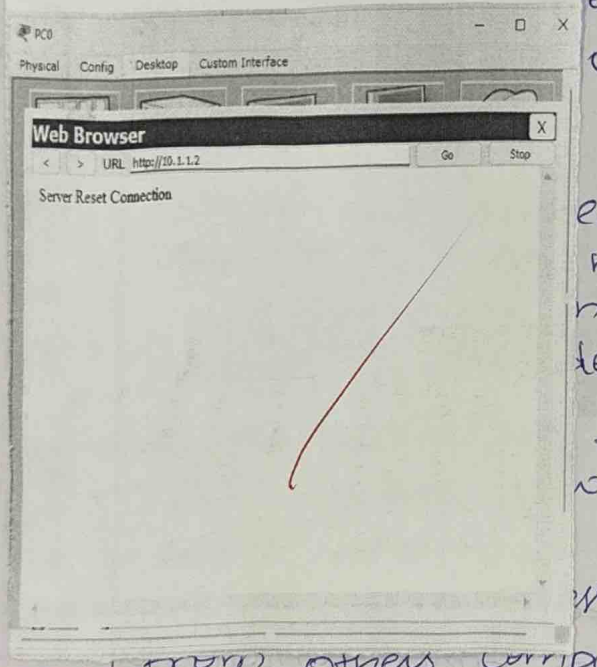
to network
et cable,
which is a simple as plugging one end of the ethernet cable.

• Step 4:
Assign IP address to your PCs.

1) Log on to the client computer as administrator or as owner.
2) Click Network and internet connections.

3) Righ click local area connection -> click on properties -> ollowing ip address ip address





ork switch.
omputer to the swit
switch's web intefac
eb inteface: This show
page for the switch
settings.
ers as 10.1.1.5. Subnet
0
ectivity between
machine by using
the command
device.
folder -> click shari
with everyone on the
ss the shared folder
from others computers of the network

*Result:

Thus, Setting up & configuring LAN using switch & ethernet cables done successfully.

Ex.No.5 Experiments on Packet capture tool: Wireshark

* Aim:-
Experiments on Packet capture tool: Wireshark

=> Packet Sniffer:
- Sniffs messages being send/received from by your computer.
- store & display the contents of the various protocol fields in the messages.

=> Packet sniffer structure Diagnostic Tools
- Tcpdump:
  eg: tcpdump -ehx host 10.129.41.2 -w exe3.out
- wireshark
  - wireshark -r exe3.out

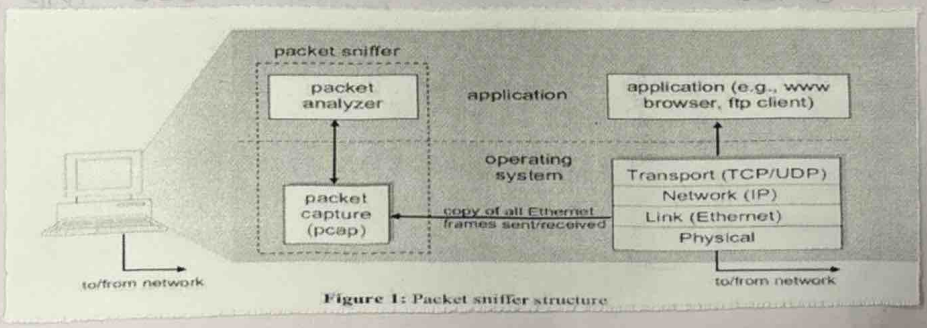


Figure 1: Packet sniffer structure

* Description:
- Wireshark:
  A network analysis tool formerly known as ethereal, captures packet in real time & display them in human-relatable format. You can use wireshark to inspect a suspicious program's network traffic.

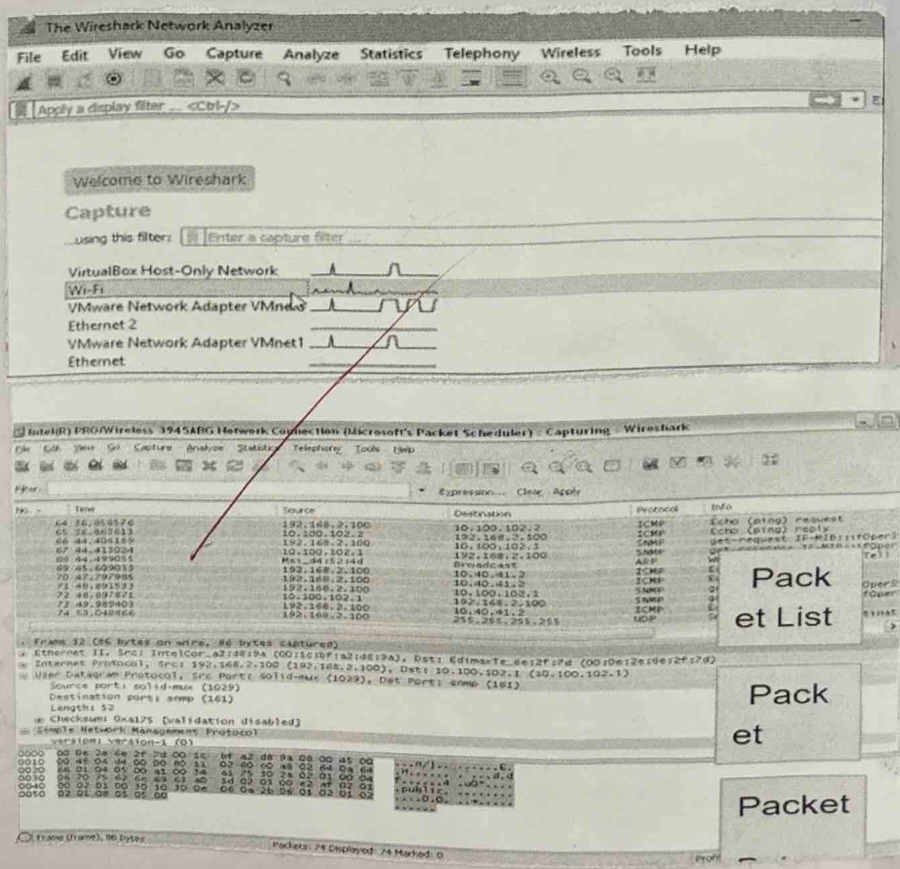
-> what we can do with wireshark:
- capture network traffic.
- watch smart statistics.
- Analyse problems.

⇒ Wireshark used for:

- Network administrators: troubleshoot network problems.
- Network security: examine security
- Developers and people.

⇒ Capturing packets:

- After downloading & installing Wire launch it & double click the nam of a network interface under capture to start capturing packets.
- As soon you click the interface's na you'll see the packets start to app in real time.
- By have promiscuous mode enabled its enabled by default. to check, click capture > options & verify the "Enable promiscuous mode on all interfaces" checkbox is activated at the bottom of this window.

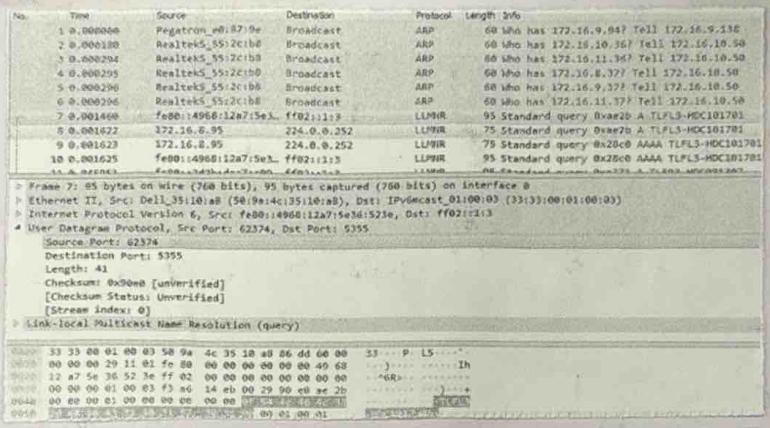


* capturing & analysing Packets using Wireshark tool :-

- To filture, capture, view, Packets in wireshark tool.
- Capture 100 packets from the ethernet : IEEE 802.3 LAN

→ procedure :
- Select Local area connection in wireshark.
- Go to capture option.
- Save the packets.

→ O/P :-



b) create a filter to display only TCP/UDP Packets, inspect the Packets & provide the flow graph.

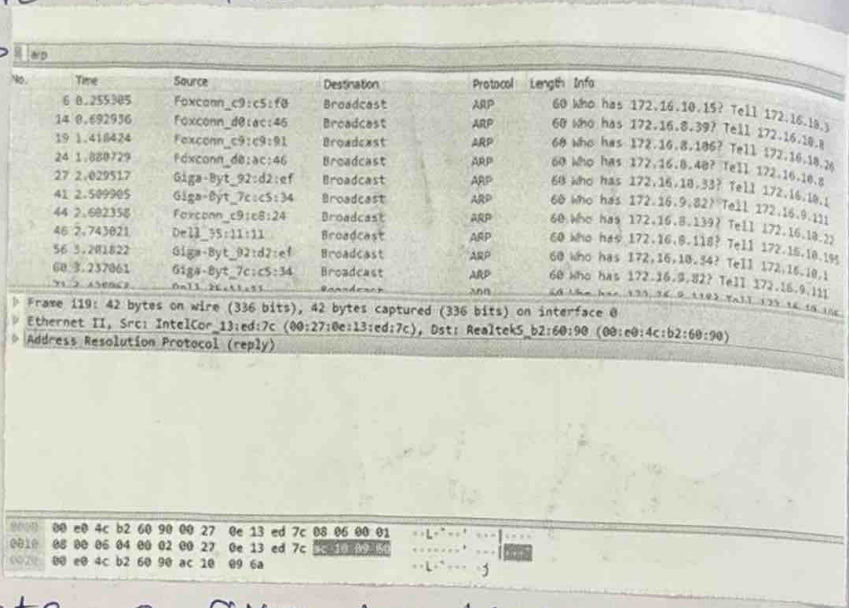
→ procedure :-
- Select Local area connection in wireshark.
- Go to capture option.
- Then click start capture.
- to Save the packets.

2) create a filter to display only ARP packets.

→ procedure:

- Go to capture option.
- Select stop capture auto after 100 pa
- Search ARP packets.
- Save the packets.

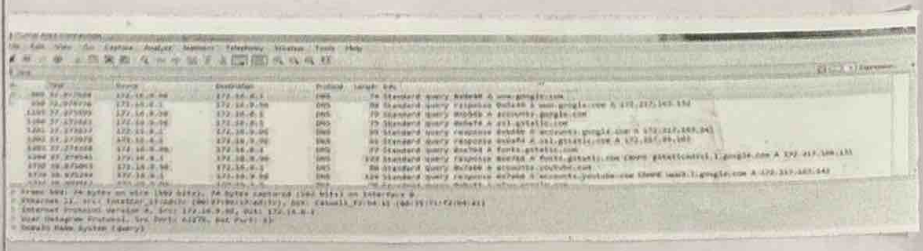
→ O/P



3) create a filter to display only DNS packets:

→ procedure:

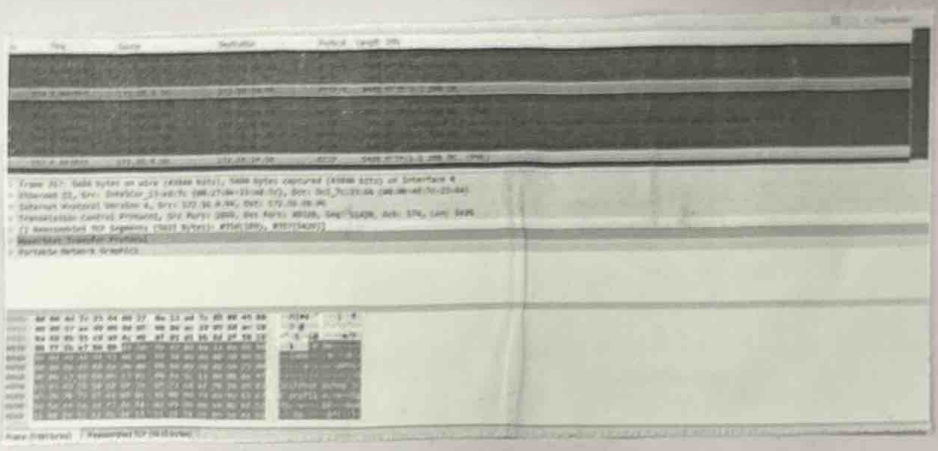
- Go to capture option.
- Select stop capture auto after 100 pack
- then click start capture.
- Search DNS packets.
- To save the packets.



4) Create a filter a display only HTTP packets:

=> procedure:

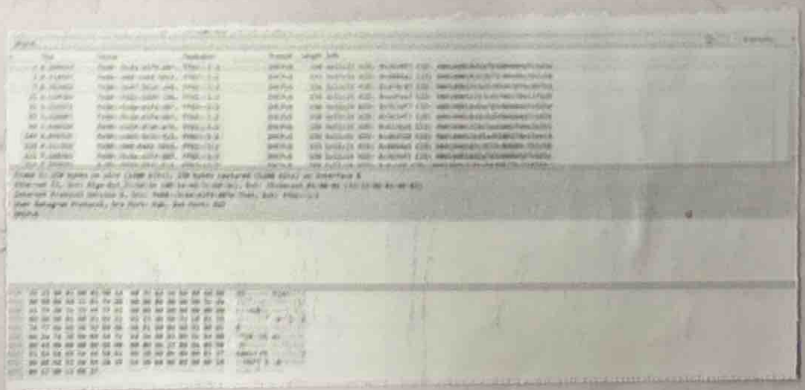
- Select Local area connection in wireshark
- Select stop capture.
- Then click start capture.
- Search HTTP packets.
- Save.



5) create a filter to display only IP/ICMP packets:

=> procedure:

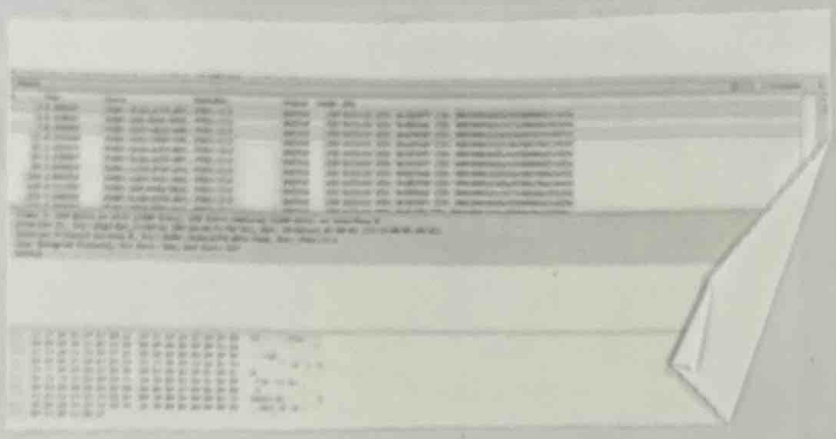
- Select Local Area connection.
- Select Stop capture.
- Then click start.
- Search ICMP/IP packets.



6) Create a filter to display only DHCP.

-> Procedure :-

- Select Local area connection.
- Go to capture option.
- Then click on start.
- Search DHCP.
- Save.



* Result :

Thus, Packet capture tool: wireshark is experimented successfully.

Ex. No. 6

Write a program to implement error detection and correction using HAMMING code concept

* Aim:-
To write a program to implement error detection and correction using HAMMING code concept.

* Error correction at Data Link Layer:-
Hamming code is a set of error-correction codes that can be used to detect. It is a technique developed by R.W. Hamming for error correction.

* Create sender program with below features:-
1) Input to sender file should be a text of any length.
2) Apply humming code on the binary code & add redundant bits to it.
3) Save this output in a file called channel

* Create a receiver program with below feautres:
1) Receiver program should read the input.
2) Apply hamming code.
3) If there is an error, display the position of the error.
4) Else remove the redundant bits & convert the binary data to ascii.

* Code:-

```
def calc_redudant_bits(m):
    for r in range(m+1):
        if(2**r) >= (m+r+1):
            return r
def main():
    data=input("Enter data bits:").strip()
    m=len(data)
    r=calc_redudant_bits(m)
    n=m+r
```

```
print(f"No. of redundant bits needed: {r}")
print(f"Total frame length: {n}")
code = ['0'] * (n+1)
j = 0
for i in range(1, n+1):
    if (i & (i-1)) != 0:
        code[i] = data[j+1]
        j += 1
for i in range(r):
    parity_pos = 2**i
    val = 0
    for pos in range(1, n+1):
        if (pos & parity_pos) != 0 and pos != parity_pos:
            if code[pos] == '1':
                val ^= 1
if change == "yes":
    received_str = input("Enter corrupted data").strip()
    if len(received_str) != n:
        print("Error: received data must be exactly {n} bits long!")
        return
    else:
        received_code = list(received_str)
if __name__ == "__main__":
    main()
```

* O/P :

```
Enter data bits : 101100
No. of redundant bits needed: 4
Total frame length (m+r): 10
Data to be transmitted: 1011100011
Transmitted bit change? (YES/NO): yes
Enter the corrupted Data Stream: 1011101011
Verification result: Error detected!
Wrong bit is at pos: 4
Corrupted data streams: 1011100011
```

* Result :

Thus, the Hamming code was implemented successfully for error detection.

Ex. No. 7

Write a program to implement flow control at data link layer using SLIDING WINDOW PROTOCOL

* Aim:-

To write a program to implement flow control at data link layer using sliding window protocol.

* create a sender program with following features:-

1) Input window size from the user.
2) Input a text message from the user.
3) consider 1 charac / frame.
4) create a frame [Frame no.; DATA]
5) send the frames [Print O/P on screen]
6) wait for the acknowledgement.
7) Reader a file called Receiver_Buffer.
8) check ACK field for Acknowledgement no.

* create a receiver file :-

1) Reader a file called Sender_Buffer.
2) check the frame no.
3) If the frame no. are as expected, write the appropriate ACK NO. in the Receiver_Buffer file. Eles write NACK no. in the Receiver_Buffer file.

* program:-

```
import time
message=input("Enter the text msg:")
window_size=int(input("Enter the size:"))
error_frame=int(input("Enter frame :"))
if window_size<=0:
        print("Invalid size")
        exit()
print("SLIDING WINDOW PROTOCOL")
print("window size:",window_size)
start=0
expected_frame=1
```

```
error_induct = False
while start< len(message):
    end=min(start+window_size, len(mess
    print("SENDER BUFFER")
    sender_buffer = []
    for i in range(start, end):
        frame_no = i+1
        data = message[i]
        print("Frame_no=", frame_no, "Data
print("In sending frames...")
time.sleep()
print("RECEIVER")
error_found = False
if error_found:
    start = expected - frame - 1
    time.sleep()
    continue
else: print("\nIncorrect Ack")
    print("waiting for correct Ack")
print("Transmission completed")
```

* O/P:

Enter the text message: hello

Enter the window size: 3

SLIDING WINDOW PROTOCOL

message: hello

window_size: 3

SENDER BUFFER

Frame_no=1 Data=h

Frame_no=2 Data=e

Frame_no=3 Data=l

RECEIVER

Received frame: 1 Data: h

Frame 1 is correct

Received frame 2 Data: e

Frame 2 is correct.

Received frame: 8 Data 8

ERROR DETECTED

expected frame: 3

Received frame: 8

* Result:

Thus, flow control of data link using sliding window is done successfully.

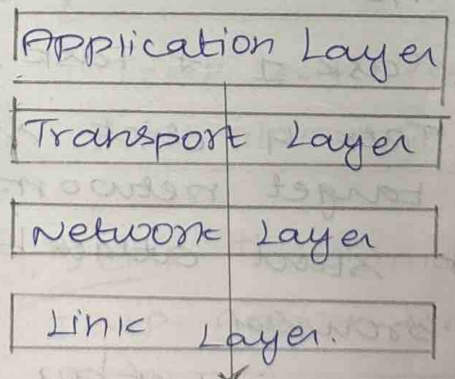
Ex. No.8 Nmap Live host discovery using TryHackme

* Aim:
To discover live hosts on a network using Nmap host-discovery scans. ARP scan, ICMP & TCP/UDP ping scan & to understand which protocol operates at which layer of the TCP/IP model.

* Introduction:

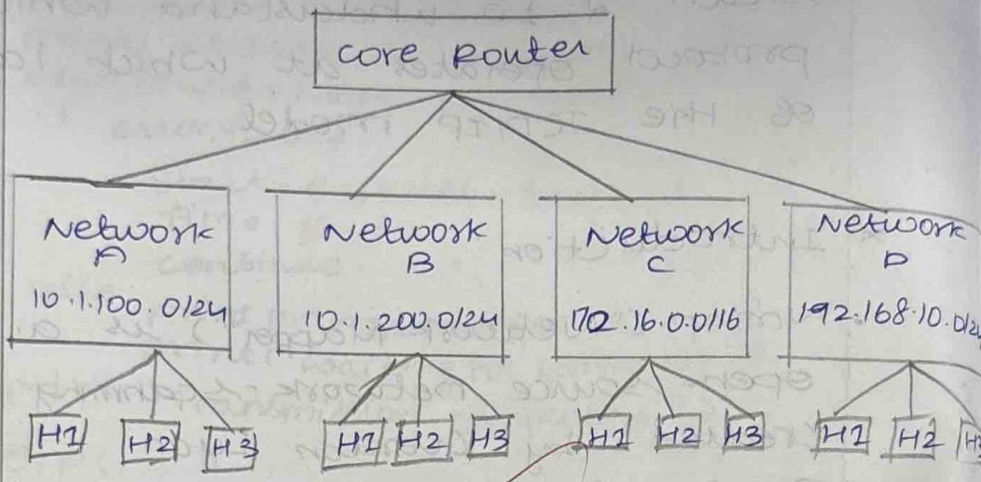
- Nmap ("Network Mapper") is a free, open-source network-scanning tool created by Gordon Lyon.
- Before Nmap can usefully scan ports on a target, it first needs to know whether the target is even online.

=> ARP (Link Layer) - for host on the same local subnet
- ICMP (Network Layer) - echo, timestamp.
- TCP (Transport Layer) - SYN + ACK ping
- UDP (Transport Layer) - relies on ICMP. Probes.



* Network segments & Subnetworks.

It is a group of computers connected by a shared physical medium. In short a segment is a physical connection, a subnet is a logical one.



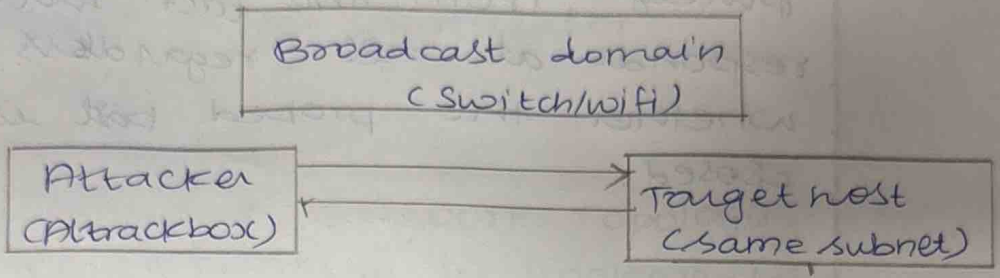
* Target Specification in Nmap.
  - A list nmap 10.10.15.
  - A range nmap 10.11.12.15.20
  - A Subnet/CIDR: nmap 10.10.1.0/24
  - A file of targets: nmap -iL list_of_hosts

* Lab setup on TryHackme.

1) Open the room & log in to your account.
2) Read Task 1 & Task 2.
3) In Task 4 click start to deploy the target network.
4) Click start attack box to launch the browser.
5) Once the attack box has booted, open a terminal inside it.
6) Note the connection IP.

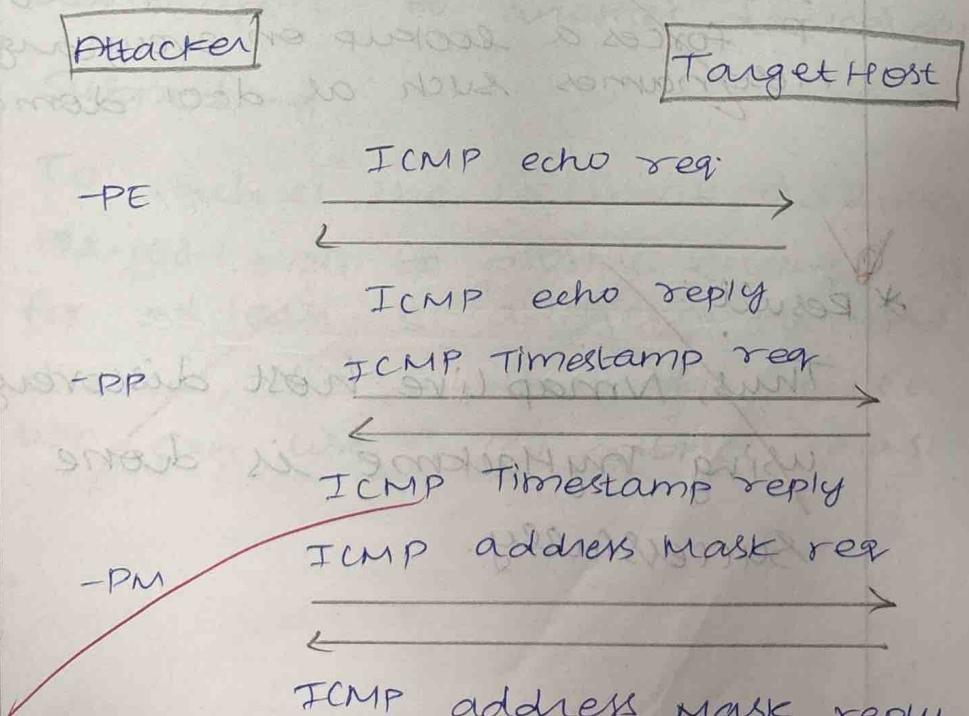
* ARP Host discovery:

On a local ethernet or wifi, communicat requires the destination's MAC addres. A reply proves the host is up



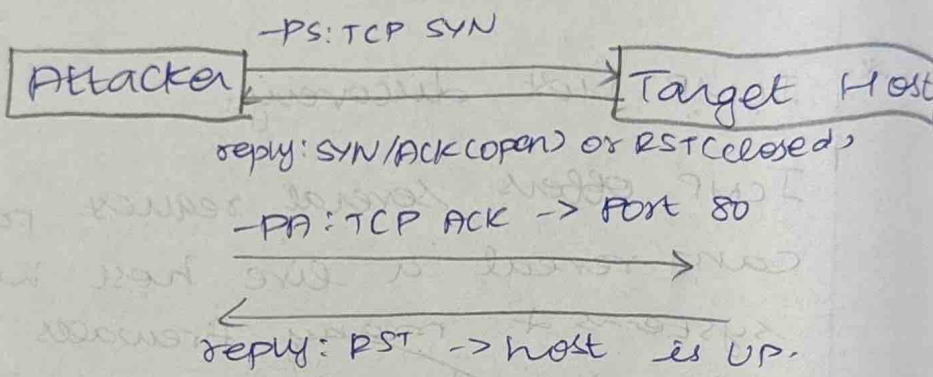
* ICMP Host discovery:

ICMP offers several request pairs that can reveal a live host. Windows systems & many firewalls block ICMP echo by default. Remember that on your own subnet, an ARP exchange still happens.



* TCP & UDP ping discovery :-

When ICMP is filtered, Nmap can send crafted TCP or UDP packets & infer that a host is alive purely from the fact that it responds at all - regardless of whether the probed port is open or closed.



* Reverse-DNS LOOKUP :

reverse DNS (rDNS) resolves an IP address back to a hostname. Nmap performs lookups on live hosts by default; -R forces a lookup on every target. Hostnames such as dc01.domain.loc

* Result :

Thus, Nmap live host discovery using TryHackme is done successfully.

Ex. No.9 Implementation of SUBNETTING in Cisco Packet Tracer.

* Aim:-

To implement subnetting in cisco Packet tracer simulator.

* Creating a Network Topology:-

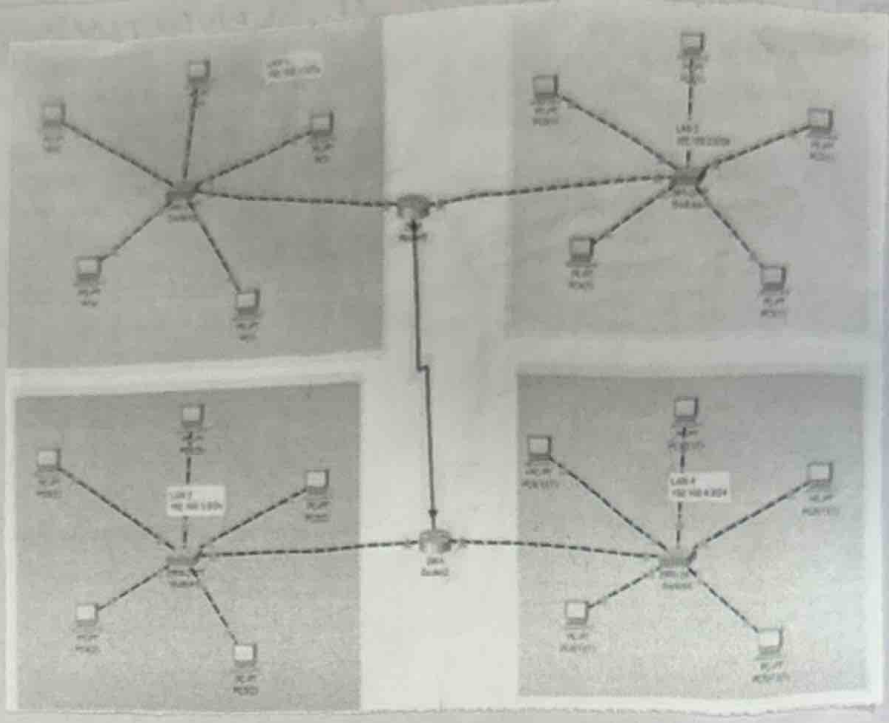
The first step in implementing classless IP subnetting is to create a network topology in packet tracer. This will create a blank network topology.

* Adding the devices:

Once we have created our network topology, we can add devices to it. Here, we will be adding routers, switches & PCs. Then, connect the devices by draging a cable from one device's port to another device's port.

* Subnetting:

To subnet the network address of 192.168.1.0/24 to provide enough space for at least 5 addresses for end devices, the switch & the router. We can use a /27 subnet mask.



⇒ The IP addressing for the network shown in the topology as follows

- Router R1:
- Gigabit Ethernet 0/0: 192.168.1.1
- Gigabit Ethernet 0/1: 192.168.2.1
- Switch S1:
- Fast Ethernet 0/1: 192.168.1.0/27
- PC1: 192.168.1.11
- PC2: 192.168.1.12
- PC3: 192.168.1.13
- PC4: 192.168.1.14
- PC5: 192.168.1.15
- Fast ethernet 0/2: 192.168.2.0/27
- PC1: 192.168.2.11
- PC2: 192.168.2.12
- PC3: 192.168.3.13
- PC4: 192.168.4.14
- PC5: 192.168.5.15

* configuring the devices:

- Now that we have added our devices & connected them, we can start configuring.

- The CLI for the router.

```
#enable
#configure terminal.
#interface Fast ethernet 0/0
#ip address {IP address} {Subnet mask}
#no shutdown
#exit.
```

- To configure gigabitethernet.

1) right click on the router & select CLI
2) Enter the following.

```
commands: enable.
configure terminal.
interface gigabit Ethernet 0/0
ip address {ip address} {Subnet mask}
no shutdown
exit.
```

* Testing the network:-

open a cmd prompt on each PC & try to ping the other PC. If the ping is successful, then the network is functing properly. We can also use the "ping" command to test connectivity between the router & the PC's.

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic
	Successful	PC4(2)	Router2	ICMP		0.000	N
	Successful	PC4(2)	PC2(1)(1)	ICMP		0.000	N
	Successful	PC0	Router0	ICMP		0.000	N

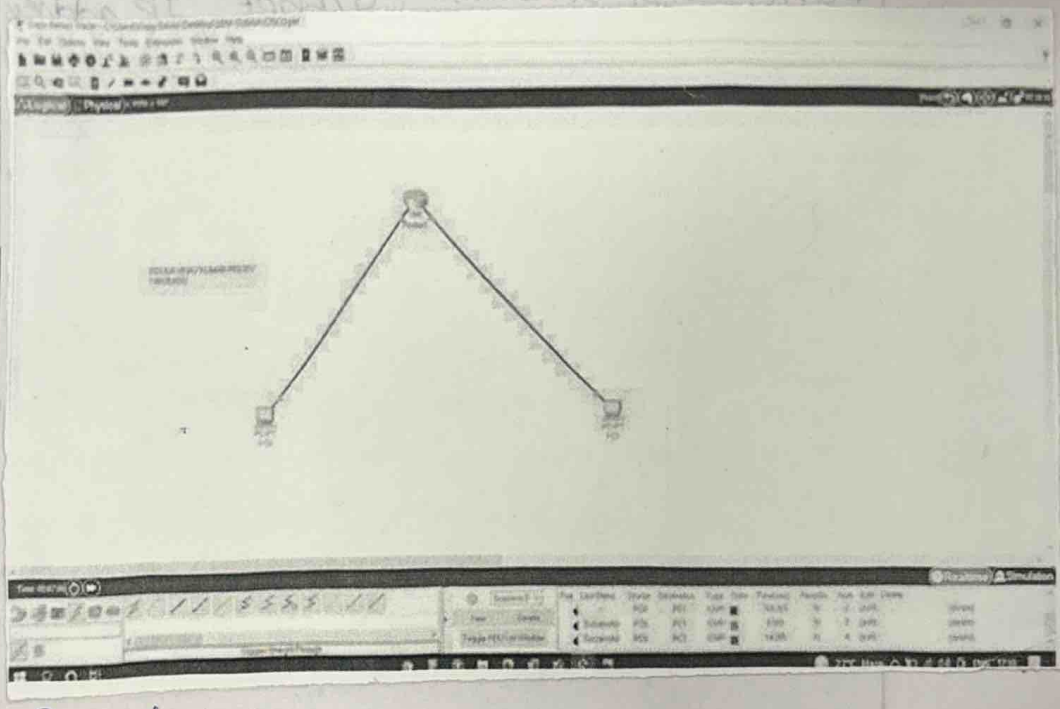
* Result :

Thus, the implementation of SUBNETTING in Cisco packet Tracer is done successfully.

EX. NO:10 Internetworking with routers in CISCO PACKET TRACER

(a) Design & configure a simple internetwork using a router.

In this network, a router & 2PCs are used. computers are connected with routers using a copper straight-through cable.



* Procedure:

=> Step-1 (Configure Router1):

1) Select the router and open CLI.
2) Press ENTER to start configuring Router1
3) Type enable to activate the privileged mode.

=> Step 2 (Configuring PCs):

1) Assign IP address to every PC in the network.
2) Select the PC, Go to the desktop & select IP configuration & assign an IP address, Default gateway.
3) Assign the default gateway of PC0 as 192.168.10.1

=> Step 3 (connecting PCs with router):

1) connect FastEthernet 0 port of PC0 with FastEthernet 0/0 port of router using a copper straight-through cable.

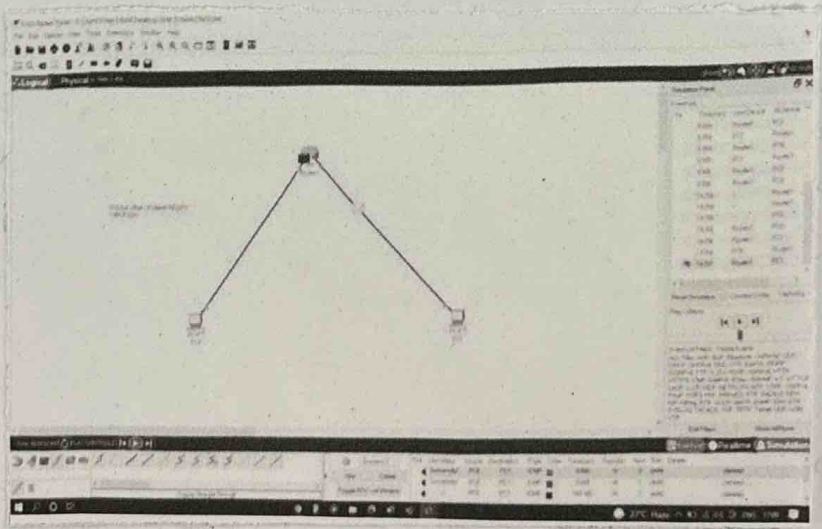
2) connect FastEthernet 0 port of PC1 with FastEthernet 0/1 port of Router using a copper straight-through cable.

=> Router configuration Table:

Device Name	IP address FastEthernet 0/0	Subnet Mask	IP address FastEthernet 0/1	Sub Ma
Router1	192.168.10.1	255.255.255.0	192.168.20.1	25 25

=> PC configuration Table:

Device Name	IP address	Subnet Mask	Gate
PC 0	192.168.10.2	255.255.255.0	192.16
PC 1	192.168.20.2	255.255.255.0	192.16 20.1



* Step 2: Configure the Laptop:-

(a) Configure the Laptop to access the wireless network.

- Click on the Laptop icon on the Packet Tracer Logical workshop.
- In the physical tab you will need to remove the ethernet copper module & replace it
- To do this, you first power the Laptop off by clicking the power button on the side of the laptop.
- Click on the Desktop tab at the top of the Laptop configuration window.
- Select the network, click on the connect tab found below the site information pane.

* Step 3: Configure the PC :-

(a) Configure the PC for the wired network:-

- Click on the PC icon on the Packet tracer Logical workspace & select the Desktop tab & then IP Configuration icon.
- In the IP configuration window, select the DHP radio button. close the IP configuration window.
- Click on the command prompt icon. Verify that the PC has received an IPv4 address by issuing the ipconfig/all command from the command prompt.

=> Step 4: Configure the internet cloud.

a) Install network modules if necessary.

b) Identify the from & to ports.

c) Identify the type of provider.

=> Step 5: Configure the cisco.com server

(a) Configure the cisco.com server as DHCP server.

(b) Configure the cisco.com server global.

(c) Configure the cisco.com server FastEthernet0 Interface settings.

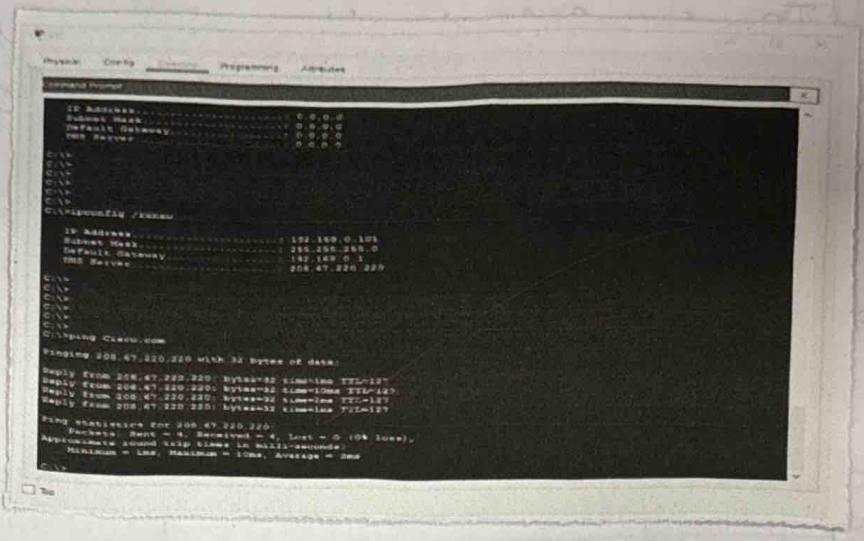
=> Part 3: Verify connectivity.

(a) Verify that the PC is receiving IPv4 Configuration information from DHCP

- Click on the PC on the Packet Tracer Logical workspace and then the select the DESKTOP.
- Click on the command prompt icon.
- In the command prompt refresh the IP settings by issuing the commands ipconfig /release & then ipconfig /renew. The o/p should show that the PC has an IP address in the 192.168.0.x range.

(b) Test connectivity to the cisco.com server from the PC:-

- From the command prompt ping cisco.com. It may take a few seconds for the ping to return. Four replies should be received.



* Result:

Thus, the Internetworking with routers in Cisco Packet Tracer is done.

Ex.no.11 Simulation of RIP using CISCO PACKET TRACER.

* Aim:-

To simulate routing information protocol (RIP) using cisco packet tracer & observe dynamic routing between different network.

* IP configuration:

Device	Interface	IP configuration
PC0	FastEthernet	10.0.0.218
Router0	Fa 0/1	10.0.0.1/8
Router0	S0/0/1	192.168.1.254/30
Router 0	S0/0/0	192.168.1.249/30
Router 1	S0/0/0	192.168.1.250/30
Router 1	S0/0/1	192.168.1.246/30
Router 2	S0/0/0	192.168.1.245/30
Router 2	S0/0/1	192.168.1.253/30
Router 2	Fa 0/1	20.0.0.1/30
Pc1	Fast Ethernet	20.0.02/30

* Procedure:

1) create the topology with 3 routers & 2Pcs.
2) connect the routers using serial cables & connect Pc's to the respective routers.

3) Assign IP addrers is all PC's & router interface.

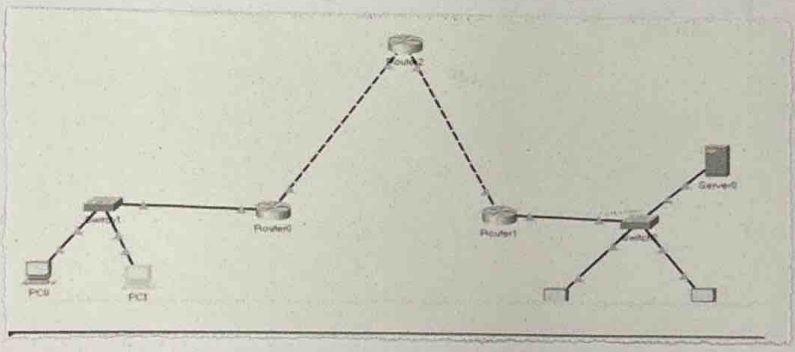
4) Enable the router interface.

5) configure the remaining interface according to the given IP table.

6) From PC0, using ping 20.0.0.2 to verify.

7) use tracer to observe the route token by packets.

8) Disconnect the direct router 0-router 2 click & observe the RIP routes. the traffic through router 1.



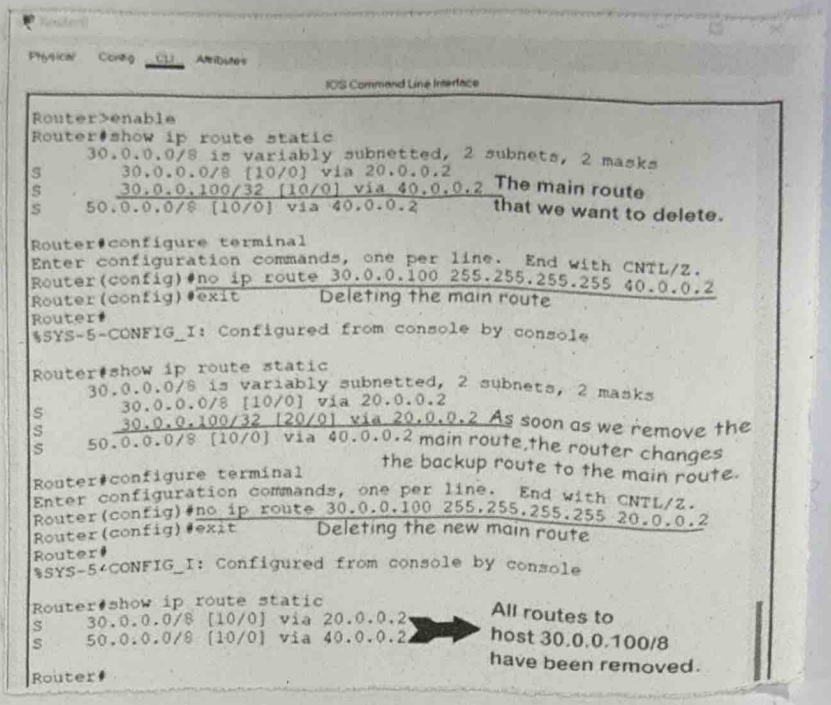
Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#ip route 30.0.0.0 255.0.0.0 20.0.0.2 10 Primary route
Router(config)#ip route 30.0.0.0 255.0.0.0 40.0.0.2 20 Backup route
Router(config)#ip route 30.0.0.100 255.255.255.255 40.0.0.2 10 Primary route
Router(config)#ip route 30.0.0.100 255.255.255.255 20.0.0.2 20 Backup route
Router(config)#ip route 50.0.0.0 255.0.0.0 40.0.0.2 10 Primary route
Router(config)#ip route 50.0.0.0 255.0.0.0 20.0.0.2 20 Backup route
Router(config)#exit
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

Router adds only primary routes to the routing table.



* Result:

Simulation of RIP using cisco packet tracer for a network is implemented successfully.

Ex no.12 End-End communication at Transport Layer.

(a) Implement echo client server using TCP/UDP sockets.

⇒ TCP

```
import socket
import threading

def start_server():
    server = socket.socket(socket.AF_INET,
                           socket.SOCK_STREAM)
    server.bind(('127.0.0.1', 5000))
    server.listen()
    print("[SERVER] Listening on 127.0.0.1:5000")

    conn, addr = server.accept()
    print(f"[SERVER] connected by {addr}")
    while True:
        data = conn.recv(1024).decode()
        if not data or data.lower() == 'quit':
            print("[SERVER] connection closed
                  by client.")
            break
    conn.close()
    server.close()

def start_client():
    client.connect(('127.0.0.1', 5000))
    print("[CLIENT] connected to server")
    while True:
        msg = input("Enter message (or 'quit' to
                    exit): ")
        client.send(msg.encode())
        if msg.lower() == 'quit':
            break
        response = client.recv(1024).decode()
    client.close()
```

```
if __name__ == "__main__":
    server_thread = threading.Thread(target
        = start_server, daemon = True
    server_thread.start()
    start_client()
```

* O/P:

```
[SERVER] Listening on 127.0.0.1:5000 [CLIENT]
                                    Connected to Serv
[SERVER] Connected by ('127.0.0.1', 53458) Ent
                                    message (or 'quit to exit
hello
[SERVER] Received: hello
[SERVER RESPONSE] Server received: hello
Enter message (or 'quit' to exit): quit
[SERVER] connection closed by client.
```

=> UDP :-

```
import socket
import threading

def udp_server():
    s = socket.socket(socket.AF_INET, sock
                      SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on
          127.0.0.1:12000")
    while True:
        data, addr = s.recvfrom
                     (1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] client
{addr} ended the connection.")
            break
        print(f"[SERVER] Received
from {addr}: {msg}")
```

```
def udp_client():
    c = socket.socket(socket.AF_INET, socket.
                      SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg = input("Enter message (or
                    'quit' to stop) :")
        c.sendto(msg.encode(), server)
        print("Echo from server:"
              data.decode())
    c.close()
    print("[CLIENT] Stopped.")

if __name__ == "__main__":
    threading.Thread(target
    =udp_server, daemon=
    True).start() udp_
    client()
```

*O/P :-

Enter message (or 'quit' to stop):[SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670):hi

Echo from server:hi

Enter message(or 'quit' to stop):quit.

[CLIENT] closing connection...[SERVER]client ('127.0.0.1', 49610) ended the connection.

[CLIENT]Stopped. [SERVER] Stopped.

*Result:

Thus, Implementation echo client server using TCP/UDP sockets.

Ex.No:13

Implement ping program.

*Aim:-

To implement your own ping program.

*program:

```
import os
import platform
import subprocess.
def ping(host="google.com", count=
        param="-n" if platform.system
                lower()=="windows" else
        command=["ping", param, str(co
                                host]
        try:
            output=subprocess.check_outp
            (command, universal_newlines=
            print(output)
        except Exception as e:
            print(f"ping failed: {e}")
if __name__=="__main__":
    ping("8.8.8.8")
```

* O/p:

Pinging 8.8.8.8 with 32 bytes of data:

Reply from 8.8.8.8 bytes=32 time=6ms
TTL=119

Reply from 8.8.8.8 bytes=32 time=9ms
TTL=119

Ping statistics for 8.8.8.8:

Packets: Sent=4, Received=4, Lost=0 (0% loss),

Approximate round trip times in milli-seconds: Minimum=5ms, Maximum=9ms
Average=7ms

* Result:

Thus, the ping program is executed successfully.

Ex.no:4 code using RAW sockets to implement Packet sniffing

*Aim:

To write a code using RAW sockets to implement Packet sniffing.

*program:-

```
from scapy.all import sniff
from scapy.layers.inet import IP,TCP,UDP,
def packet_callback(packet):
    if IP in packet:
        ip_layer=packet[IP]
        protocol=ip_layer.proto
        src_ip=ip_layer.src
        dst_ip=ip_layer.dst

        protocol_name=""
        if protocol==1:
            protocol_name="ICMP"
        elif protocol==6:
            protocol_name="TCP"
        elif protocol==17:
            protocol_name="UDP"
        else:
            protocol_name=
                "unknown Protocol"
        print(f"Protocol: {protocol_na
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dst
        print("-"*50)

sniff(iface="Wi-Fi",prn=packet_callback,
      filter="ip",store=0)
```

* O/P:

C:\Users\tkala\Downloads>python Packet.py

Protocol: TCP
Source IP: 51.155.141.117
Destination IP: 192.168.1.6

Protocol: TCP
Source IP: 192.168.1.6
Destination IP: 51.155.141.117

Protocol: UDP
Source IP: 192.168.1.6
Destination IP: 192.168.1.1

Protocol: UDP
Source IP: 192.168.1.1
Destination IP: 192.168.1.6

*Result:

Thus, code using RAW sockets to implement packet sniffing is done & executed successfully.

Completed
13/10/25